# Actividad 1:

1. Load an image containing **at least three different objects from different classes** using `cv2.imread` .

In [20]:
```
# Write your code here...
import cv2
from matplotlib import pyplot as plt
image_bgr = cv2.imread(r'H:\MaestriaUIDE\clase_vision_computador_3\data\images.jpg'
print('Image shape:', image_bgr.shape)
# Convert BGR to RGB for displaying with matplotlib
image_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)

# Display the image
plt.imshow(image_rgb)
plt.show()
```

```
Image shape: (194, 259, 3)
```

2. Draw **different colored bounding boxes** around each object (at least one of the bounding boxes should be yellow). Use a **line thickness** of 5 for both bounding boxes.

In [21]:
```
# Write your code here...

# Perro (AMARILLO)
cv2.rectangle(image_bgr, (10, 30), (120, 230), (0, 255, 255), 5)
```

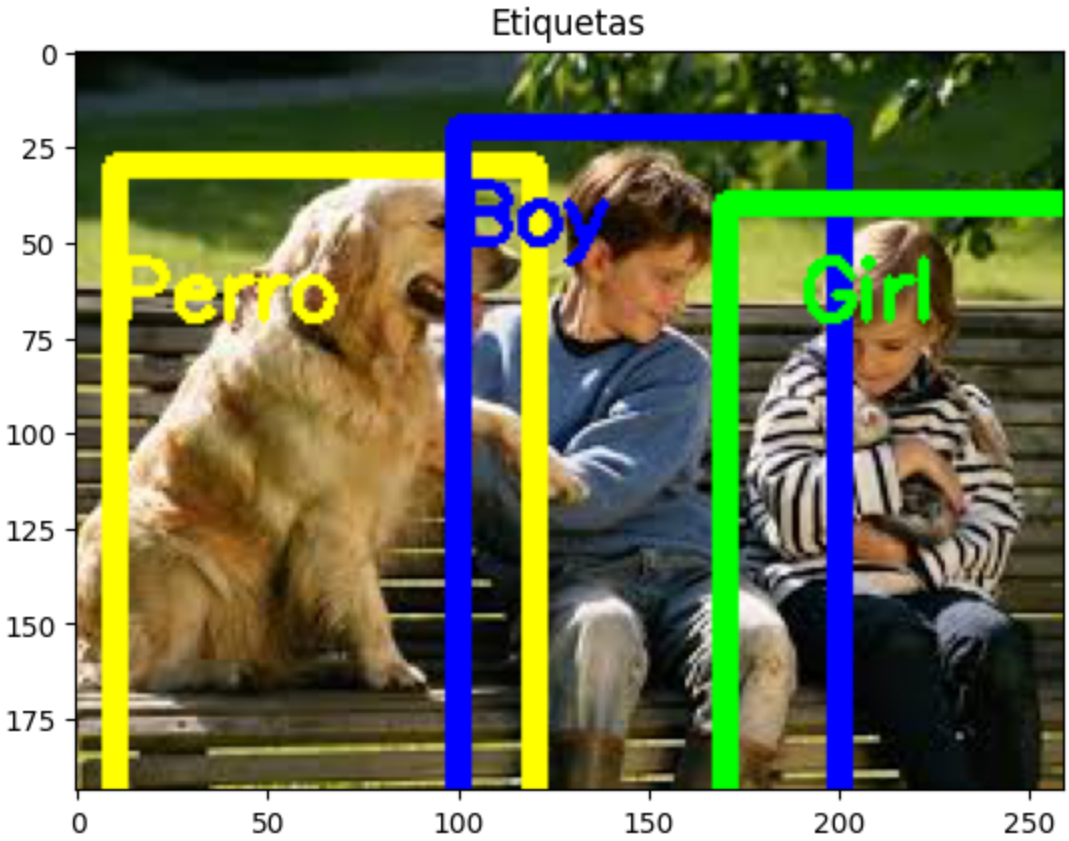
```
# Niño (AZUL)
cv2.rectangle(image_bgr, (100, 20), (200, 230), (255, 0, 0), 5)

# Niña (VERDE)
cv2.rectangle(image_bgr, (170, 40), (280, 230), (0, 255, 0), 5)

# Convertir BGR → RGB
img_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)
```

3. Add **text labels** above each bounding box (e.g., "Cat", "Ball", etc.) using `cv2.putText` .

In [22]:
```
# Write your code here...

#Perro (AMARILLO)
cv2.putText(image_bgr, "Perro", (10, 70), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 2

# Niño (AZUL)
cv2.putText(image_bgr, "Boy", (100, 50), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 0, 0)

# Niña (VERDE)
cv2.putText(image_bgr, "Girl", (190, 70), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0

# Convertir BGR → RGB
img_rgb = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2RGB)
```

4. Display the **resulting image**.

In [23]:
```
# Write your code here...
# Mostrar imagen


# Mostrar imagen
plt.imshow(img_rgb)
plt.title("Etiquetas")
#plt.axis("off")
plt.show()
```




5. Suggest one method (written, no code) to **detect objects automatically** in an image (no manual coordinates). Name the model or technique you would use and briefly explain how it produces bounding boxes.

Write your answer here...

Una buena opción es usar el modelo YOLO, el cual nos permite detectar objetos en tiempo real este detecta y marca las coordenadas automaticamente lo hace mediante cuadros delimitadores lo cual hace que este método sea rápido, eficiente.

## Grading Rubric:

| Criteria | Points | Description |
|---|---|---|
| **1. Image Loading** | 1 | Successfully loads an image with at least three objects using `cv2.imread`. Correct file path and proper handling of image object. |
| **2. Bounding Boxes** | 3 | Draws **different colored bounding boxes** around each object. Boxes have correct size and position, and line thickness is set to 5. |
| **3. Text Labels** | 2 | Adds **accurate text labels** above each bounding box using `cv2.putText`. Labels are readable and positioned correctly. |

| Criteria | Points | Description |
|---|---|---|
| **4. Automatic Detection Suggestion** | 4 | Provides a clear and reasonable method to detect objects automatically. Shows understanding of dynamic object detection. |

Total: 10 points

**Prepared by:**

Manuel Eugenio Morocho-Cayamcela